

HACK HYDRA

Build with the HydraDB open-source repo.

BUILD WINDOW

Aug 12-20, 2026

FORMAT

Online

TEAM SIZE

1 to 4 people

PRIZE POOL

\$10,000

Nine days. Three hard problems. Build something that works.

HACK HYDRA WEBSITE

hackhydra.hydradb.com

REGISTER (FREE)

luma.com/h038glzk

Welcome to Hack Hydra

HydraDB is a graph database built for AI applications, using object storage to stay fast and cost-efficient. From August 12-20, 2026 we are open-sourcing HydraDB and inviting developers globally to build across three technical tracks: enterprise knowledge, coding agents and agent memory, while competing for up to \$10,000 in prizes.

Every track is a problem where relationships, structure and connected context actually matter, and none of them is something a vector database handles well. Pick one, build with HydraDB, and submit something working by August 20.

AT A GLANCE

BUILD WINDOW

August 12-20, 2026

TEAM SIZE

1 to 4 people

FORMAT

Online

PRIZE POOL

\$10,000

SUBMISSION DEADLINE

August 20, 11:59 PM PT

WHAT A COMPLETE PROJECT LOOKS LIKE

This is the definition of done. If a line is missing, the judges cannot properly evaluate your work.

- Uses HydraDB meaningfully
- Has a working implementation
- Includes a 3-minute demo video
- Is submitted through the official Google Form
- Addresses one of the challenge tracks
- Includes a public GitHub repository
- Ships with an open-source license

Nine days to build

Everything happens online. The only fixed point that really matters is the submission deadline.

AUGUST 12

Hackathon begins

The HydraDB open-source repository goes live at github.com/hydradb/hydradb. Choose your track and start building.

AUGUST 12-20

Build

Nine days to ship. The HydraDB team is active in [Discord](#) throughout for questions, repo walkthroughs, technical support and office hours.

AUGUST 20

Final submissions due

Submit your Google Form, public GitHub repository and demo video by 11:59 PM PT.

AFTER AUGUST 20

Judging

The HydraDB team and the judging panel review every eligible submission, first within its track and then across all three.

TO BE ANNOUNCED

Winners announced

Track Winners, the Grand Champion, the Overall Runner-Up, Third Place and Best Use of HydraDB are announced once judging is complete. The exact date goes out in [Discord](#) and on the hackathon page.

IMPORTANT

Participant-authored development on your submitted project must begin on or after **August 12, 2026**.

Three problems. Pick one.

Choose the track that best matches what you want to build. Each one is a graph problem first: if a vector index could answer it cleanly, it is not the interesting version of the question. The top submission from every track advances to the final round. Full briefs follow on the next three pages.

01 Enterprise Context + Ontology

Turn roughly half a million messy documents from nine business tools into a clean, queryable ontology, then answer real questions against it.

HARD PART: ENTITY RESOLUTION AND ONTOLOGY ALIGNMENT

02 Repos, Dependencies + Code as Graphs

Model npm or PyPI as a graph and compute the blast radius of a compromised package, or build a code graph that beats similarity search for IDE assistants.

HARD PART: TRAVERSAL AT ECOSYSTEM SCALE, UNDER TIME PRESSURE

03 Memory + Context Retrieval

Build an agent memory layer for long-running conversations. 30 to 40 sessions, 115,000 tokens a question, facts that change and facts that are simply not there.

HARD PART: CHRONOLOGY, OVERWRITES AND CORRECT ABSTENTION

YOU CAN ENTER MORE THAN ONE

Teams may submit to several tracks, as long as each entry is a meaningfully distinct project, and you can switch tracks mid-hackathon as long as the final submission fits the one you enter it under.

DATASETS AND UPDATES

The suggested datasets are optional unless a track says otherwise. Bring your own, and disclose it in the README. Full links and FAQ: hackhydra.hydradb.com

Enterprise Context + Ontology

CORE IDEA

Build an ontology out of real enterprise applications. You get roughly half a million documents drawn from nine different sources, with all the noise an actual company has: misfiled documents, near-duplicates, and statements that flatly contradict each other.

Your job is to turn that into a clean, queryable ontology in HydraDB and then answer questions ranging from simple lookups to multi-hop reasoning, conflict resolution, and correctly recognising when the answer just isn't in there.

Slack

Gmail

Linear

Google Drive

HubSpot

Fireflies

GitHub

Jira

Confluence

THE HARD PART

Extraction is the easy part now that LLMs are cheap. The hard part is entity resolution and ontology alignment: deciding that “Sam”, “@soham” and “S. Ratnaparkhi” are one person, and figuring out which of two contradictory statements to trust.

DATASETS

Enterprise RAG Bench

github.com/onyx-dot-app/EnterpriseRAG-Bench

Salesforce HERB

huggingface.co/datasets/Salesforce/HERB

WHAT STRONG WORK SHOWS

- A graph schema that survives contact with the noise
- Precise resolution of aliases, duplicates and near-matches
- Multi-hop answers with a traceable path through the graph
- Saying “not in the data” instead of inventing an answer

Repos, Dependencies + Code as Graphs

Supply-chain attacks via npm and PyPI are surging, and current developer tools fail to give real-time, deep context on malicious dependencies. This is fundamentally a graph traversal problem, not a semantic similarity problem.

WHEN A PACKAGE IS COMPROMISED, ANSWER

- Which internal services are transitively exposed?
- Which apps resolved the bad version while it was live?
- Are there likely typosquat packages nearby?
- Which version introduced the vulnerability?
- Which packages share maintainers or infrastructure?
- What is the complete blast radius?

A Supply chain blast radius

In the TanStack compromise this May, 84 malicious package artifacts were published across 42 packages within six minutes of the CI pipeline being breached. The worm went on to hit Mistral AI, UiPath and over 160 other npm and PyPI packages, self-propagating and persisting in `.claude/` and `.vscode/` directories in a way that survived `npm uninstall`.

The defender's problem is speed. When a package is compromised at 09:00, which of your services are exposed by 09:06? That is a transitive reverse-dependency closure over an ecosystem graph with tens of millions of versioned nodes, and it is the kind of question a vector index cannot answer at all.

Build the npm or PyPI dependency graph in HydraDB and answer it. Then go further: which packages share a maintainer with the compromised one, which lockfiles resolved to the bad version while it was live, which names are close enough to a popular package to be a typosquat.

EVALUATION

- Ground truth is free from OSV and the GitHub Advisory Database
- Recent advisories are held out, and teams get an older ecosystem graph
- Scored on precision, recall, query latency and cost

B Code graphs for IDE assistants

Every IDE assistant embeds repositories and retrieves chunks by similarity. Similarity is a weak proxy for relevance. Code needs call chains, type definitions, startup configs and tests.

Build a code graph in HydraDB that serves better context than similarity search alone. This is the missing piece for editor wrappers today.

EVALUATION

- SWE-bench
- Retrieval heuristics against a similarity-only baseline

CHOOSE A OR B

Pick one direction and finish it. Both are judged on the same track.

Memory + Context Retrieval

CORE IDEA

Make your own memO, and ace the benchmarks. Build an agent memory layer for cross-session continuity.

Your system processes chat histories spanning 30 to 40 sessions and 115,000 tokens per question. It has to synthesize facts across sessions, keep chronological order, and track information that was overwritten along the way.

Facts spread across many sessions

Chronological reasoning

Information that changes over time

Overwritten facts

Missing information

Correct abstention

THE HARD PART

Long-context models drop 30 to 60% in accuracy here, mostly by failing at abstention. Knowing when the answer simply isn't in the history, and saying so instead of inventing one.

DATASETS

LongMemEval

github.com/xiaowu0162/LongMemEval

LongMemEval V2

github.com/xiaowu0162/LongMemEval-V2

BEAM

github.com/mohammadtavakoli78/BEAM

WHAT STRONG WORK SHOWS

- Accuracy that a vector store alone cannot reach
- A memory graph that makes time and revisions explicit
- Abstention handled as a first-class result, not an edge case
- Read and write cost that would survive real usage

What you need to submit

Three things, all due August 20 at 11:59 PM PT: the submission form, a demo video of 3 minutes or less, and a public GitHub repository.

01 Google Form

Complete the official Hack Hydra submission form. It asks for:

- Project name and short description
- The problem you are addressing
- What you built
- Deployed project link, if you have one
- How your project uses HydraDB
- Tech stack
- Team members and contributions
- GitHub repository and demo video

SUBMISSION FORM

forms.gle/GiMYKxLj9zPQcqqc8

02 3-minute demo video

Four things, in order:

The problem. What are you trying to solve?

The project. What did you build?

The demo. Show it working.

HydraDB. Where is it used, and why does it matter?

YouTube links are fine. Unlisted videos are fine as long as judges can open them. Anything after the 3-minute mark may not be reviewed.

03 Public GitHub repository

- Complete source code
- An open-source license
- No participant-authored commits before August 12, 2026
- A clear README
- Setup and run instructions
- An explanation of how you use HydraDB
- Dependencies and environment information
- Attribution where required

Judges have to be able to open the repo and understand what you built, without requesting access.

Keep it simple

Start on August 12, use HydraDB properly, and keep everything open to inspection.

Build it during the nine days

Work on your submission begins on or after August 12, 2026. Reusing general-purpose libraries, frameworks and templates is fine, as long as the project you submit was substantially built during the hackathon.

Use HydraDB properly

HydraDB has to be meaningfully integrated into the project and contribute to core functionality. Be ready to say where it is used and what the project would lose without it.

Teams of 1 to 4

Every member has to be listed on the submission form, and you can only be on one team.

Multiple tracks are allowed

Teams may submit to multiple tracks, but each submission must be a meaningfully distinct project. A team may receive only one Track Winner prize, and stays eligible for Best Use of HydraDB either way.

Open-source tools are allowed

Libraries, frameworks, APIs, public datasets and AI coding assistants are all fair game. Credit anything you borrowed in the README. Commit history inherited from upstream repos or dependencies does not count against you.

Keep the repository public and licensed

Judges have to be able to open it without requesting access, and it needs an open-source license.

Submit by August 20

Everything goes through the official form by 11:59 PM PT. Late submissions are not accepted unless HydraDB announces an official extension, so open your own repo, video and demo links before you submit.

A SUBMISSION MAY BE DISQUALIFIED FOR

- Work started before August 12, 2026
- Missing or private GitHub repository
- Duplicating the same project across tracks
- HydraDB not used meaningfully
- Violating the rules or the code of conduct
- Submitting substantially pre-existing work
- No open-source license in the repository
- Missing demo video
- Late submission

\$10,000 in prizes.

Four awards. One incredible opportunity.

\$5,000

Grand Champion

Awarded to the best overall submission across all three tracks.

\$3,000

Overall Runner-Up

Awarded to the second best overall submission across all three tracks.

\$1,500

Third Place

Awarded to the third best overall submission across all three tracks.

\$500

Best Use of HydraDB

Awarded to the submission with the most thoughtful and technically interesting use of HydraDB.

- The top submission from each track advances to the final round. Final placement is determined across all three tracks.

BEST USE OF HYDRADB COULD LOOK LIKE

- A particularly strong graph data model
- A novel retrieval or reasoning approach
- An interesting use of relationships, traversal or context
- A use case that is hard to pull off with vector or relational approaches

Judged separately, and open to any eligible submission including a finalist.

ELIGIBILITY

- A complete submission by the deadline: form, repo and video
- A team can be a finalist in more than one track, but takes home only one Track Winner prize
- Every team stays eligible for Best Use of HydraDB, whichever track they enter
- HydraDB employees and organizers may build along, but are not eligible for prizes

You keep ownership of what you build, subject to your open-source license and the third-party tools you used. HydraDB reserves the right not to award a prize if submissions do not meet the minimum quality bar.

Judged in your track first, then across all three.

01

Within your track

Projects are first evaluated against the other submissions in the track they were entered in. The top submission from each track advances.

JUDGES CONSIDER

- Technical execution
- Use of HydraDB and graph-native approaches
- Product completeness and usability
- Quality of results
- Originality

02

Final round

Judges compare the three finalists holistically and rank them for Grand Champion, Overall Runner-Up and Third Place.

A STRONG SUBMISSION HAS

- A functional product or demo
- Real ingestion and retrieval workflows
- A clear use case
- A thoughtful technical implementation

We care about working, thoughtful products, not just benchmark scores.

Before you submit

Check every item. Most disqualifications are a missing link, not a weak project.

- | | |
|---|--|
| ☐ Submission form is complete | ☐ GitHub repository is public and licensed |
| ☐ README explains the project clearly | ☐ Setup instructions actually work |
| ☐ Demo video is 3 minutes or less | ☐ Video link is accessible |
| ☐ Deployed project link works, if you gave one | ☐ HydraDB usage is clearly explained |
| ☐ Team members are listed correctly | ☐ All your work began on or after August 12 |
| ☐ Submitted before 11:59 PM PT on August 20 | |

FINAL ADVICE

Stop adding features before the deadline. Test what you already built.

Everything you need

Every link is clickable in this PDF and scannable from a printed copy.



Hack Hydra website

hackhydra.hydradb.com

Briefs, datasets, rules, FAQ and updates.



Project submission form

forms.gle/GrMYKxLj9zPQcqc8

One per project, closes August 20.



Hack Hydra Discord

discord.gg/D8cGSa9H9

Support, announcements, team formation.



HydraDB documentation

docs.hydradb.com

Start here when you are building.



HydraDB research

research.hydradb.com

Benchmarks and architecture.



Hack Hydra on Luma

luma.com/h038glzk

Registration and event updates.



HydraDB open-source repo

github.com/hydra-db/hydradb

Clone this and start building.



HydraDB dashboard

dashboard.hydradb.com

Manage your instance and keys.

HYDRADB WEBSITE

hydradb.com

NEED HELP?

Ask in [Discord](#). The HydraDB team is there throughout for technical questions, office hours, repo support and submission clarifications.

Build small. Test early. Ship something real.